MARCH2026

AMTCS

IoT project on

Automating Measurements Using a Temperature Controller and Spectrometer: Initially Simulated

(client: Tyndall National Institute)

FINAL REPORT

NAME: Marina Zubialevich

STUDENT ID: 20108904

Supervisor (SETU): Dr. Anita Kealy

Mentor (Tyndall N.I.): Dr. Pietro Pampili

MODULE: PROJECT

COURSE: Higher Diploma in Computer Science

Declaration of Authenticity

I declare that the work which follows is my own, and that any quotations from any sources (e.g. books, journals, the internet) are clearly identified as such by the use of 'single quotation marks', for shorter excerpt and identified italics for longer quotations. All quotations and paraphrases are accompanied by (date, author) in the text and a fuller citation is the bibliography. I have not submitted the work represented in this report in any other course of study leading to an academic award.

Student Date .....28/03/2026.....

Work Place Mentor Date .....29/03/2026.....

1. Contents

Declaration of Authenticity ..... **ii**
Table of Figures ..... **v**
Acknowledgements ..... **vi**
AI Use Declaration ..... **vii**
List of Abbreviations ..... **viii**
Executive Summary **1**
1. Introduction **2**
2. Stakeholders and Personas **3**
Users 3
User Requirements 4
3. Feature Analysis **5**
Core system features: 5
4. Market and Technology Analysis **6**
Market Analysis 6
Repository Analysis 6
5. System Architecture and Models **7**
Instrumentation 7
System Model 7
Pipeline Model 9
Data Model 9
6. Minimum Viable Product (MVP) Definition **10**
7. Optional Extensions **11**
8. Methodology **12**
Development Methodology 12
Development Plan 12
Conclusion for RAMP 12
9. Sprint 2 – System architecture and communication framework **14**
Initial Setup and Development Environment Preparation 14
GUI Development 14
Raspberry Pi Configuration and Communication Setup 16
PLC Communication Architecture 17
Rationale for the Chosen Architecture 18
SDK Hardware Key Issue 18
Conclusion and MVP Progress at the End of Sprint 2 19

10. Sprint 3 – TC integration and temperature stabilisation logic 20
Experiment Parameters Exchange 20
Experiment State Model 20
Recovery Mechanism (optional) 20
Simulation and Production Modes 21
Conclusion and MVP Progress at the End of Sprint 3 21
11. Sprint 4 – Monochromator and CCD Integration 22
SDK Analysis and Integration 22
Hardware Connection and Initialisation 22
Spectral Window Handling and Multi-Window Acquisition 23
Real Acquisition Workflow 23
Conclusion and MVP Progress at the End of the Sprint 4 24
12. Sprint 5 – System Orchestration, Logging and Error Handling 25
Startup 25
Graceful Shutdown 25
Orchestration Structure 25
Pre-Experiment Stage 26
Experiment Stage 26
User Interface and User Experience (UX) Improvements 27
Logging and Progress Tracking 28
Conclusion and MVP Progress at the End of Sprint 5 28
13. Sprint 6 – System Testing, Validation and Refinement 29
Extended Experiment Control 29
System Recovery and Fault Tolerance 29
Cosmic Ray Artefact Removal 30
Auto exposure feature 30
Testing and Refinement 31
Conclusion and MVP Progress at the End of Sprint 6 31
Conclusion 32
Reflection 32
Key Skills 32
Challenges 32
Future Work 33
References 34

Table of Figures

Figure 1 – AMTCS System schematic 8

Figure 2 – Three provisional tabs of PC C++ GUI (as discussed with stakeholders): *a*) Hardware Connectivity/Initialisation, *b*) Experiment Settings, *c*) Experiment Progress Flow 8

Figure 3 – Project development plan 13

Figure 4 – Connectivity Window – for managing device connectivity 15

Figure 5 – Experiment Settings Window – for entering experiment parameters 15

Figure 6 – Experiment Flow Window – for displaying experiment status and messages 16

Figure 7 – Schematic of the multithread organisation of PLC. 17

Figure 8 – 'Experiment Flow' screen showing typical logs during experiment execution. 28

Figure 9 – Logs after UI App recovery (simulated crash). 29

Figure 10 – Example logs during measurement subroutine (UI App terminal). 31

Acknowledgements

I would like to express my sincere gratitude to Dr. Pietro Pampili for his continuous guidance throughout this project. His expertise in spectroscopy and his detailed explanations of the required experimental settings were invaluable in shaping the system requirements and ensuring that the project remained aligned with real laboratory needs. I am also grateful to Dr. Vitaly Zubialevich for his support, for clarifying laboratory constraints, safety considerations, long-term maintainability requirements and, especially, for his assistance in timely resolving the spectrometer SDK hardware-key issue with equipment vendor engineers.

I would also like to thank my SETU advisor Colm Dunphy for his help in reviewing project proposal and my SETU project supervisor Dr. Anita Kealy for her advice and constructive feedback on the structure and preparation of this Final Report. Her guidance helped me refine the presentation of my work and maintain clarity throughout the document.

AI Use Declaration

During the preparation of this report, I used several generative AI tools, including Microsoft Copilot, ChatGPT and Google Gemini, as supportive writing and learning aids. These tools were used to clarify C++ syntax and programming concepts, help me reason through architectural decisions such as the multi-threaded PLC design and the structure of the experiment state model, and improve the clarity and structure of written English (of this report). All technical decisions, system designs, implementation work, and conclusions are my own. AI-generated suggestions were critically reviewed, adapted, and verified by me before implementing in the project code or main text of this report.

List of Abbreviations

Abbreviation	Meaning
AMTCS	Automated Measurement Temperature Control and Spectrometer
ASCII	American Standard Code for Information Interchange
CCD	Charge-Coupled Device
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
MVP	Minimum Viable Product
OS	Operating System
PC	Personal Computer
PLC	Programmable Logic Controller
PL	Photoluminescence
RAMP	Research, Analysis, Modelling and Planning
RPi	Raspberry Pi
SDK	Software Development Kit
TC	Temperature Controller
VM	Virtual Machine
VS2015	Visual Studio 2015

Executive Summary

This report presents the design, development, and implementation of the laboratory automation project “AMTCS – IoT project on Automating Measurements Using a Temperature Controller and Spectrometer, Initially Simulated (client: Tyndall National Institute)”. It provides an overview of the full project lifecycle, including the Research, Analysis, Modelling and Planning phase, system architecture design, and subsequent implementation stages (sprints).

The report summarises the work carried out across individual sprints and describes the step-by-step implementation of key system features. These include the user interface, PLC logic structure, communication between components, temperature stabilisation logic, integration of SDK-controlled hardware, overall system orchestration, optimised spectrometric measurements, and logging. It also presents system validation under real laboratory conditions and discusses the achieved results.

Finally, the report reflects on the overall project outcomes, evaluates the system against the initial requirements and MVP definition and outlines possible future work.

1. Introduction

This project focused on automating temperature-dependent spectroscopic measurements in the PL/Raman Spectroscopy Laboratory at Tyndall National Institute.

Previously, experiments required manual temperature control, monitoring temperature stabilisation, triggering measurements and data collection. This made experiments time-consuming and increased risk of human error, especially during long measurement sessions.

The goal of the project was to design an automation system that integrates laboratory instruments into a coordinated workflow. The system controls temperature changes, detects when the temperature becomes stable, automatically triggers spectrometer measurements, collects experimental data, and saves them as files.

This automation allowed experiments to run with minimal user interaction, improving efficiency and reducing operator workload. Future work includes enabling remote experiment monitoring and user intervention only when necessary.

2. Stakeholders and Personas

To better understand the overall goal and specific objectives of the project, interviews were conducted with the laboratory manager and the main laboratory users. Based on these discussions, the key stakeholders were identified, along with their goals, expectations, and practical needs related to the system.

Users

Dr. Pietro Pampili (my mentor from Tyndal)

Role: Advanced User and Main Stakeholder

Dr. Pampili is the main beneficiary of the automation system. He runs advanced temperature-dependent spectroscopy experiments and requires a reliable, repeatable and efficient workflow.

Dr. Vitaly Zubialevich

Role: Laboratory Responsible and System Owner

Dr. Zubialevich is responsible for the operation and safety of the laboratory. He is concerned with system reliability, correct execution of experiments, and long-term maintainability.

PhD Students

Role: Routine Users

PhD students are daily users of the spectroscopy setup. They are less experienced and more likely to make procedural mistakes.

User	Goals	Pain Points
Dr. Pietro Pampili (PP)	• automated temperature sweeps; • reliable temperature stabilisation; • automatic measurements; • clean datasets; • ability to run long experiments without supervision	• time lost on repetitive tasks; • manual monitoring; • risk of starting at wrong temperature; • cannot multitask during long experiment
Dr. Vitaly Zubialevich (VZ)	• reduce human errors; • increase lab efficiency; • ensure reproducible experiments; • maintain stable system	• manual and repetitive experiment procedures; • inconsistent workflows; • student mistakes; • lack of logging
PhD Students (PS)	• simple workflow; • clear system status information; • easy setup • reduced risk of configuration errors	• complex configuration; • unclear experiment state; • wrong parameters; • inconsistent procedures

User Requirements

From discussion with stakeholders and further analysis, the following high-level requirements were identified that the system should:

- automate overall experiment flow (PP, PS)
  - temperature control and stabilisation detection
  - trigger spectroscopic measurements automatically
- coordinate experimental steps in a consistent and repeatable manner (PP, VZ)
- log all experiment parameters and results (VZ)
- provide clear experiment status information (PS)
- minimise the need for manual intervention (PP, PS)
- prevent invalid or unsafe experiment configurations (PP, PS, VZ)
- adjust measurements parameters according to signal intensity (PP – optional)
- detect and deal with "cosmic rays"* (PP – optional)

* Cosmic ray is rare but unavoidable occurrence of bit register error in Charge-Coupled Devices (CCD) leading to incorrect reading from individual pixels (can be few orders of magnitude higher than neighbour pixels while of random sign). This effect is believed to be induced by particles (neutrons, muons, etc.) reaching the Earth surface from the cosmos. The effect sometimes called Single-Event Upset/Bit Flip (Wikipedia Contributors, 2024).

3. Feature Analysis

Core system features:

a) Temperature controller (TC) integration via serial port communication
b) Detection of temperature stabilisation before measurements
c) Spectrometer control via manufacturer software development kit (SDK)
d) PLC-style logic to control the sequence of experiment steps
e) Automated data capture and storage
f) Experiment status monitoring
g) Error detection and logging/reporting

4. Market and Technology Analysis

Market Analysis

A review of existing laboratory automation solutions showed that many commercial platforms are proprietary and designed to operate within specific vendor ecosystems, such as Thorlabs (Thorlabs Inc., 2026) or Oxford Instruments (Oxford Instruments, 2026). While open-source tools for laboratory automation exist, for example, HORIBA EzSpec-SDK (HORIBA Scientific, 2026), they usually require significant effort to integrate instruments from different manufacturers.

This project focuses on developing a lightweight and flexible automation system that integrates existing laboratory equipment without replacing the current infrastructure. Instead of creating a general-purpose automation platform, the system is designed to solve practical problems in the PL[†]/Raman Spectroscopy Laboratory at Tyndall National Institute.

Repository Analysis

An analysis of open-source repositories on GitHub and other resources shows several relevant types of projects e.g. HORIBA EzSpec-SDK (HORIBA Scientific, 2026), including laboratory automation frameworks(OE-FET, 2020), instrument control libraries (National Instruments, 2023), serial communication tools, equipment drivers, etc.

These repositories provide useful design ideas and communication patterns. However, none of them fully meet the requirements of temperature-resolved spectroscopy using existing laboratory equipment. For this reason, the project will reuse selected concepts while implementing a custom solution adapted to the PL/Raman laboratory.

[†] PL stands for photoluminescence, a type of luminescence excited with photons.

5. System Architecture and Models

The proposed system architecture consists of:

- Raspberry Pi (RPi) acting as the main automation controller and running PLC-style orchestration logic.
- Windows PC used to control the spectrometer (iHR320 monochromator with Synapse CCD camera from HORIBA Scientific – existing lab setup).
- Serial communication with Cryo-Con 32 TC using ASCII-based commands according to the manufacturer manual (Cryogenic Control Systems Inc., 2011).
- Local Ethernet connection between the RPi and the PC via TCP.

RPi was selected because it runs a full Linux operating system, supports networking, and provides USB ports that can be converted to serial interfaces using adapters for communication with TC. It offers sufficient performance, low power consumption, and reliability for continuous operation.

According to the manufacturer's SDK documentation (Witter, 2004), compatibility was officially stated up to Windows XP. However, in practice, the original SDK-based software runs under Windows 7. Therefore, the use of Windows 7 PC as a gateway to the spectroscopic equipment was dictated by the need to preserve the existing infrastructure. This ensured continued availability of the room-temperature measurement setup using the original vendor software.

Instrumentation

The main constraint on programming language choice was imposed by the SDK for spectrometer communication. It supports development only in Visual Basic, Visual Basic for Applications, and C++ (Witter, 2004). C++ was selected because it provides greater long-term value for professional development in scientific and IoT automation. While all three languages can interface with the SDK, C++ offers direct exposure to both low-level concepts and higher-level technologies(MFC, COM), which are essential for instrument control and high-performance data acquisition.

For PLC-style control on the RPi, Python was selected. It is practical and efficient for automation workflows. In addition to prior experience, Python was chosen for its clear syntax, large ecosystem of libraries, and strong support for serial communication, threading, and networking. It runs reliably on the RPi without complex configuration, which enabled fast prototyping and testing of control logic.

Because C++ was new to me, the learning curve was steep. To reduce delays, AI was used initially to clarify syntax, understand project configuration, and resolve compilation and linking errors. This support was particularly helpful when working with header/source structure and event handling at the beginning of implementation.

System Model

The system consists of three primary components (Figure 1):

- RPi Automation Controller (Python-based PLC)
- PC UI/Instrument Gateway (C++, SDK)
- Server/mobile application for remote monitoring (optional, future work)

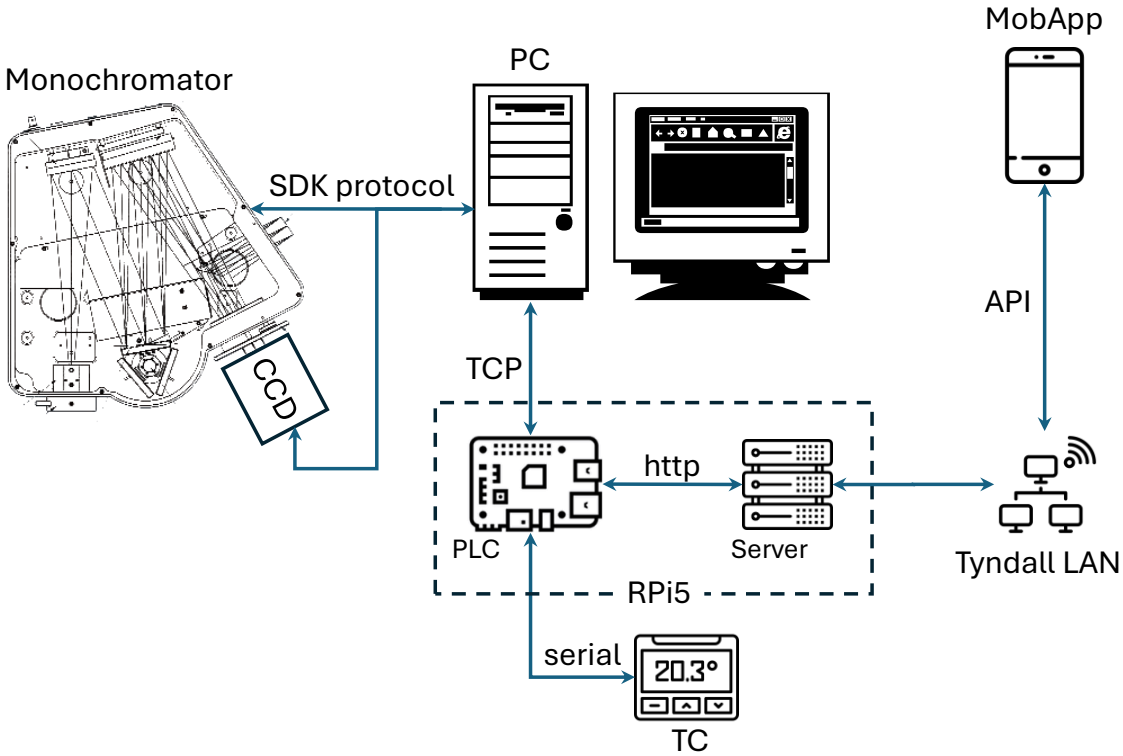


Figure 1. – AMTCS System schematic

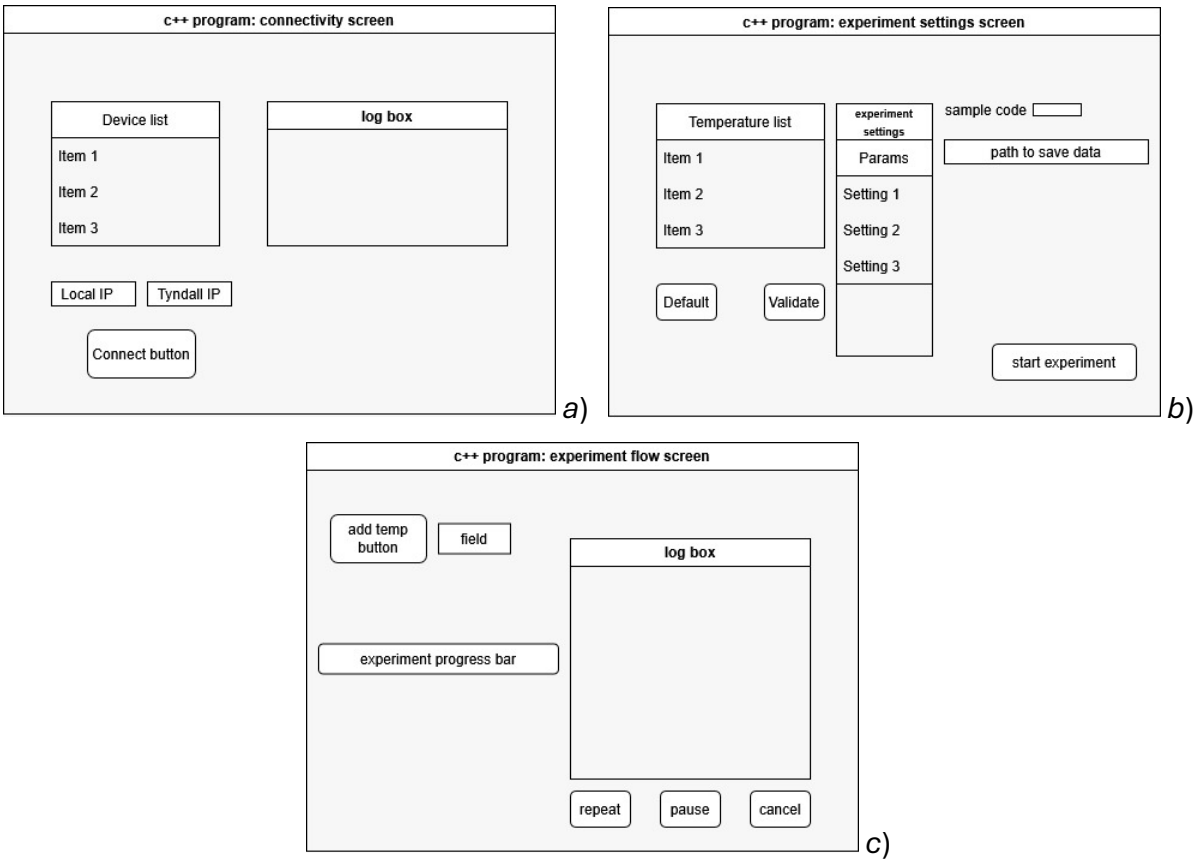


Figure 2. – Three provisional tabs of PC C++ GUI (as discussed with stakeholders): *a*) Hardware Connectivity/Initialisation, *b*) Experiment Settings, *c*) Experiment Progress Flow

The RPi communicates with the TC via serial interface and with the PC via TCP over Ethernet. The PC communicates with the monochromator and CCD camera through custom software developed using vendor SDK. Figure 2 shows the preliminary design of the PC C++ user interface, developed in collaboration with project stakeholders.

The RPi executes the experimental workflow and coordinates individual steps.

Pipeline Model

Before starting an experiment, the system checks connectivity between all components (PC, RPi, TC, and spectrometer) to ensure availability (Figure 2, *a*). After successful verification, the experiment pipeline proceeds as follows:

User sets experiment parameters and presses “Start” (Figure 2, *b*).

1. PLC receives the configuration and sets the first target temperature to the TC
2. PLC monitors temperature until stabilisation
3. PLC instructs the PC software to trigger the spectrometer measurement
4. Data are captured and stored
5. The step is logged
6. System moves to next temperature setpoint
7. Process repeats until completion

Data Model

The system data model includes:

- Experiment configuration parameters (spectral range, acquisition time, etc.)
- Temperature setpoints
- Temperature stabilisation timestamps
- File references to spectroscopic data
- System logs and error reports

6. Minimum Viable Product (MVP) Definition

MVP includes the following software components:

- Serial communication between the PLC and TC
- Temperature stabilisation detection
- SDK-based spectrometer triggering by PLC
- Experiment sequencing logic (PLC-style orchestration)
- Automated data capture and logging
- Local experiment status interface

7. Optional Extensions

The following features were identified as valuable improvements but are not included in the MVP due to time and scope constraints. They could be implemented during the internship period beyond the academic deadline:

- Detection and correction of cosmic ray artefacts using data processing
- Automatic optimisation of acquisition parameters based on real-time signal intensity
- Experiment logging and notifications on a locally hosted server (RPi)
- Mobile and/or browser-based access for remote experiment monitoring

These features extend system functionality but are not essential for the core automation workflow. The MVP focuses on delivering a stable, reliable, and maintainable automation framework first.

8. Methodology

Development Methodology

Agile development methodology was adopted. Development proceeded incrementally, starting with simulated or virtual hardware components and gradually integrating the real equipment.

The project was organised into two-week sprints. Each sprint included planning, implementation, testing, and review. Progress was tracked using a Trello Kanban board, and Git was used for version control with regular commits.

This approach helped test the main automation functions early, reduce problems during hardware integration, and connect the real equipment step-by-step.

Development Plan

The project was structured into six two-week sprints (Figure 3):

1: Research, analysis and initial design
2: System architecture and communication framework
3: TC integration and temperature stabilisation logic
4: Spectrometer SDK integration
5: System orchestration, logging and error handling
6: System testing, validation and refinement

Each sprint contained clearly defined tasks. Task status was tracked using Red (not started), Amber (in progress), and Green (completed) indicators.

Conclusion for RAMP

RAMP phase established structured and realistic plan for developing an automation system for temperature-resolved PL spectroscopy. The architecture, requirements, and methodology were defined, providing a solid foundation for development phase.

Optional extensions such as remote monitoring may be implemented if time permits.

Figure 3. – Project development plan

9. Sprint 2 – System architecture and communication framework

Initial Setup and Development Environment Preparation

Because laboratory equipment was frequently used for ongoing experiments, full access to the physical setup was not always available during the early stage of the project. For this reason, development initially took place in simulated environment.

To ensure continuous progress, a virtual machine was created on my personal computer. The VM was configured with Windows 7 and Microsoft Visual Studio 2015 (VS2015).

According to the spectrometer SDK documentation, the software requires Windows 2000 or Windows XP (Witter, 2004). However, the laboratory workstation provided by the vendor runs Windows 7, indicating practical (though undocumented) compatibility with the newer operating system. Therefore, instead of strictly following the original documentation, the actual laboratory environment was replicated, as required by stakeholders.

The oldest officially available version was VS2015, which is compatible with Windows 7. The C++ examples provided with the SDK were successfully compiled using this version after minor adjustments. For consistency, VS2015 was installed both on the laboratory PC and on the development VM. This ensured identical development and testing environments, reducing the risk of compatibility issues during integration.

This approach allowed:

- continued development outside laboratory hours,
- independent TCP testing,
- early identification of configuration issues,
- reduced delays due to limited lab access.

Setting up this controlled development environment was an important step that allowed the project to progress in a structured and stable manner.

GUI Development

To develop the graphical user interface (GUI), the most experienced users (PP and VZ) were interviewed to collect requirements for layout and functionality. The GUI, implemented in C++, allows researchers to enter experimental parameters in a simple and intuitive way and monitor experiment logs.

The following interface windows were designed and partially implemented (primarily frontend at this stage, Figure 4 - Figure 6):

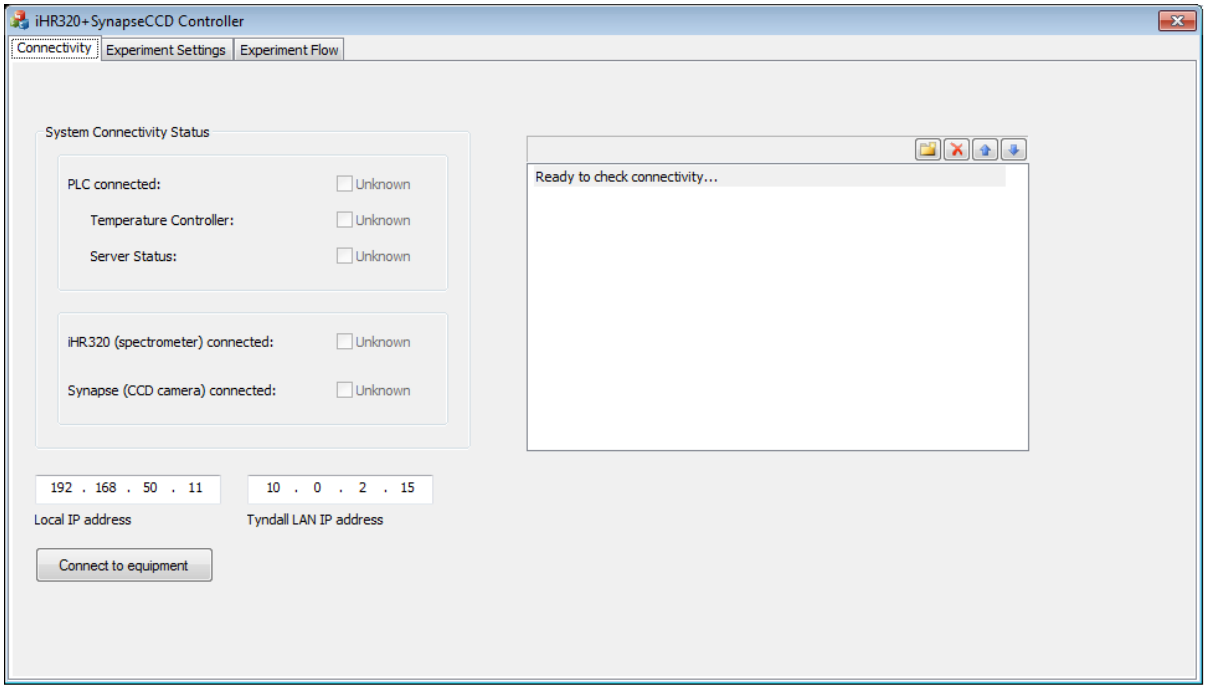


Figure 4. – Connectivity Window – for managing device connectivity

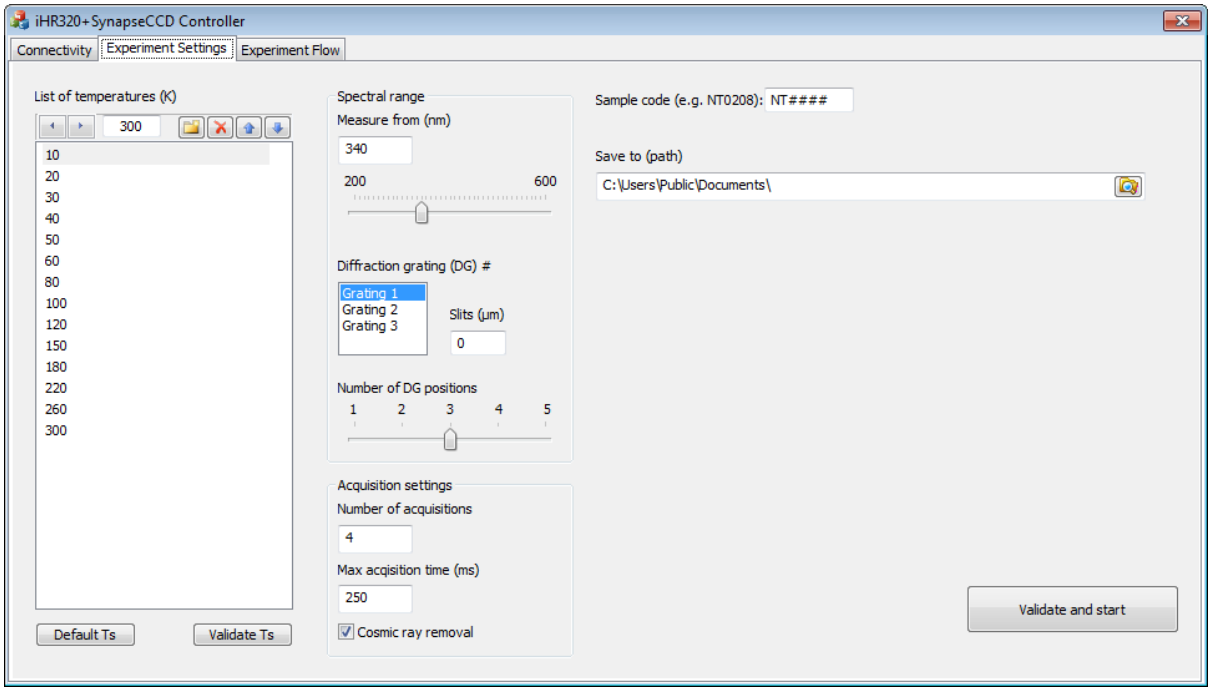


Figure 5. – Experiment Settings Window – for entering experiment parameters

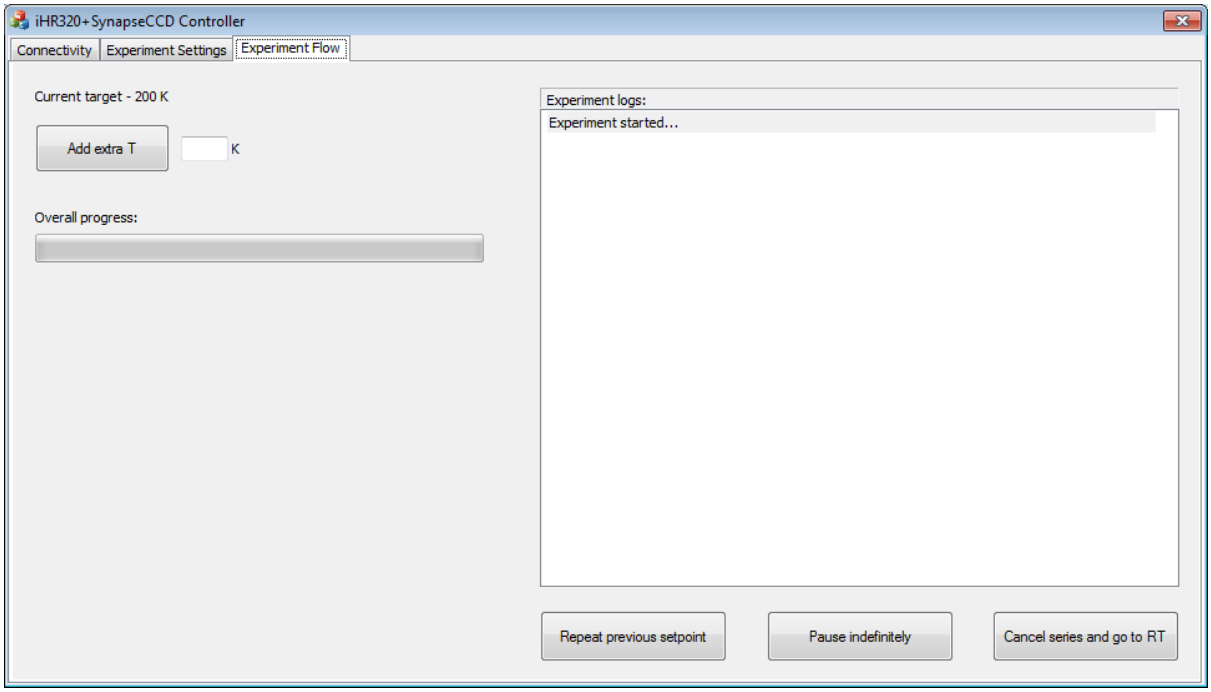


Figure 6. – Experiment Flow Window – for displaying experiment status and messages

At this stage, these windows provide the structural interface of the application, while backend functionality is still being implemented.

Raspberry Pi Configuration and Communication Setup

In parallel with the C++ application development, the RPi was configured as part of the AMTCS system.

The following connections were established:

- **RPi5 <-> Windows 7 PC** via Ethernet. The devices were configured within the same local network using static IP addressing to ensure stable communication.
- **RPi5 <-> Temperature Controller** via serial port connection. A USB-to-serial adapter was used on the RPi side. Serial communication parameters were configured according to the TC manual (Cryogenic Control Systems Inc., 2011):
    - Baud Rate: 9600
    - Parity: None
    - Bits: 8
    - Stop Bits: 1
    - Mode: Half Duplex

To verify communication between the C++ application and the RPi, a simple TCP echo server was implemented on the RPi and documented in a spike branch of the project GitHub repository (Zubialevich, 2026). On the C++ side, a *SendTCPmessage()* function was developed to test TCP data transmission.

Connectivity between the Windows 7 machine and the RPi was first verified in the local development setup and later confirmed in the laboratory environment.

By the end of this phase, communication between all main components of the AMTCS system was successfully established and tested.

PLC Communication Architecture

For the PLC component implemented on the RPi, a multi-threaded communication architecture was designed (Figure 7). Three separate threads were created for:

- serial communication with the TC
- TCP communication with the C++ application
- main logic and orchestration

Each communication channel has its own listener that continuously monitors incoming messages.

The listeners were intentionally designed as "dumb" components. They do not make decisions (except simple heartbeat checks) and contain no experiment logic. Their responsibilities are:

- listening for and interpreting incoming messages,
- placing interpreted messages into shared queues,
- sending outgoing messages when instructed by the main logic thread.

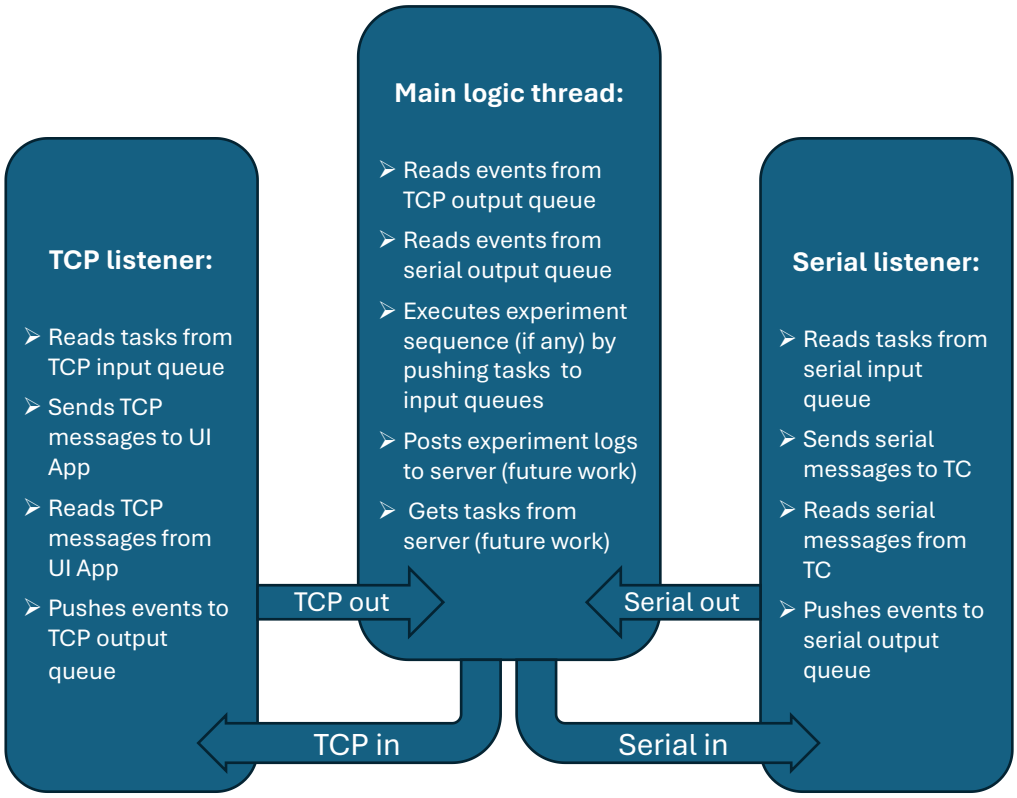


Figure 7. – Schematic of the multithread organisation of PLC.

The TCP listener receives messages on one port and sends responses on another. This separation clarifies communication flow and prevents conflicts between incoming and outgoing data streams.

All decision-making is handled by the main logic thread. The process is as follows:

1. A message is received by either the TCP or serial listener.
2. The message is placed into a queue.
3. The main program processes messages sequentially.
4. Based on the system state, it decides whether to send a command via TCP (to the C++ application), via serial (to the TC) or to wait for further data.

At this stage, the logic remains relatively simple. The main objective was to confirm that:

- Messages are correctly received.
- Messages are correctly forwarded.
- All system components communicate reliably.

Because the experiment workflow is sequential, no time-critical race conditions currently occur. Commands are processed step by step rather than through parallel decision flows, which simplifies synchronisation and reduces early-stage complexity.

This architecture ensures clear separation between communication handling and decision logic, making future extensions easier to implement and maintain.

Rationale for the Chosen Architecture

The multi-threaded architecture with “dumb” listeners and a central main thread was selected for several reasons.

First, it improves maintainability. Communication handling is separated from experiment logic, giving each component a clearly defined responsibility. This simplifies code readability, debugging and future modifications.

Second, it supports scalability. If additional hardware components are introduced (although unlikely), new communication threads can be added without major changes to the core logic.

Third, it simplifies testing. Since communication and logic are separated, parts of the system can be simulated independently. This was especially important during early development when laboratory access was limited.

Overall, this architecture provides clarity, modularity, and flexibility for future expansion.

SDK Hardware Key Issue

After adapting the system to the laboratory environment at Tyndall, an unexpected issue was identified.

Although the vendor SDK was correctly installed and the example projects were successfully compiled, they could not be executed. The system required a hardware key (USB dongle) to enable SDK access. The key available in the laboratory allowed standard spectrometer software operation but did not support SDK-level control.

As a result, the SDK examples could not run, and communication with the spectrometer through SDK examples (and foreseeably in the C++) was blocked.

The issue was resolved with support from laboratory staff and the vendor’s engineer during a remote session. The hardware key was upgraded and reconfigured to enable SDK functionality. After the update, successful spectrometer communication was confirmed using the SDK examples.

Conclusion and MVP Progress at the End of Sprint 2

By the end of Sprint 2, the development environment was fully configured and reliable communication between system components was established. Initial integration with the spectrometer SDK was also validated, confirming that the hardware could be controlled programmatically.

At this stage, the foundational elements required for the MVP were in place, particularly in terms of communication infrastructure and system architecture. These components enabled further development of temperature control logic, spectrometer integration, and overall system orchestration in subsequent sprints.

10. Sprint 3 – TC integration and temperature stabilisation logic

During Sprint 3, the focus shifted to more realistic interaction between the PLC and TC.

To continue developing the temperature controller logic without fully relying on physical hardware, the PLC needed to understand the current experiment stage. Initially, experiment state management was planned for Sprint 5 (overall orchestration and logging). However, it became clear that without defining experiment states earlier, structured development and testing of temperature control would be difficult.

Therefore, a core experiment state model was implemented during Sprint 3.

Experiment Parameters Exchange

The C++ application collects user-defined experiment parameters (Figure 5) into a structure **struct** *ExperimentParameters*. These parameters are serialized into a JSON string using the **nlohmann/json** library (Lohmann, 2021).

The JSON structure is transmitted to the PLC via TCP.

On the PLC side:

- The parameters are deserialized (using Python's standard *JSON* module).
- They are stored in a Python **@dataclass** named *ExperimentParameters*.

This class mirrors the corresponding C++ structure, ensuring consistency between both system components.

Experiment State Model

The experiment consists of multiple temperature cycles. Each cycle includes two steps: 1) temperature stabilisation; 2) spectrum acquisition.

Each step has properties: **string** *StepName* and **int** *StepStatus*. The status can take one of the three values:

- *WAITING* (0)
- *REQUESTED* (1)
- *COMPLETED* (2)

When all cycles and their steps are completed, the experiment is considered finished.

This structure is implemented in the **class** *ExperimentState* (with **class** *ExperimentFlow* as a subcomponent containing the list of cycles). It allows the PLC main thread to determine:

- the current target temperature,
- whether it has been requested,
- whether stabilisation is in progress,
- whether spectrum acquisition was requested,
- whether acquisition is completed,
- whether the experiment has finished.

Recovery Mechanism (optional)

The state model is designed to support future fault recovery. If either the RPi or the C++ application crashes, the system will be able to resume from the last completed step

rather than restarting the entire experiment. For this purpose, *ExperimentParameters* is included within the *ExperimentState* class.

Full recovery logic is planned for a later sprint, during orchestration and error-handling implementation if time permits.

Simulation and Production Modes

To enable development both inside and outside the laboratory, two experiment modes were implemented:

- *SIMULATION* – hardware behaviour is emulated;
- *PRODUCTION* – PLC communicates with the real TC via serial port.

In simulation mode, when the main thread requests the current temperature, the system generates a simulated response. The simulated temperature gradually approaches the target value, mimicking real thermal behaviour. Once the temperature stabilises within a defined tolerance range, the experiment state is updated accordingly. At this point:

- The PLC instructs the TCP thread to send a measurement request to the C++ application.
- The application responds with an acknowledgment ("*AFFIRMATIVE*").
- After a simulated acquisition delay, it sends a completion message ("*DONE*").

This handshake verified correct sequencing between the PLC and the application.

Conclusion and MVP Progress at the End of Sprint 3

By the end of Sprint 3, the PLC logic was extended to support the experiment structure, and temperature stabilisation was integrated into the experiment state model. A simulation mode was introduced, enabling development and testing without continuous access to hardware. In addition, a basic end-to-end workflow, covering temperature control and spectrum acquisition requests, was validated, including laboratory testing.

At this stage, the core experiment sequencing logic required for the MVP was established. This provided a working framework for integrating hardware and replacing simulated behaviour with actual device interaction in subsequent sprints.

11. Sprint 4 – Monochromator and CCD Integration

Sprint 4 focused on full integration of the monochromator and CCD camera using the vendor-provided SDK and working example code. This stage marked the transition from simulated behaviour to real hardware-controlled acquisition.

SDK Analysis and Integration

The SDK documentation provided by the vendor was relatively old, and in some cases the information was outdated or incomplete. As a result, integration required careful interpretation of the manual and extensive testing of the provided example code.

The SDK is based on a COM object architecture and requires proper initialisation of device "sink" interface for event handling. The device sink interface class, as recommended in the SDK manual (Witter, 2004), was taken in full from the SDK examples and only marginally adapted for the needs of my application. The rest of integrating into the existing C++ application required significant adjustments to the project configuration and careful management of object implementation and callbacks. Where possible original comments from SDK examples related to rather large chunks of code taken from example project were preserved to mark the authorship.

To improve maintainability and simplify further development, wrapper functions were implemented for the main SDK operations, including:

- Device connection and initialisation
- Setting acquisition parameters
- Triggering spectrum acquisition
- Retrieving measurement results

These wrappers isolate vendor-specific SDK calls from the rest of the application logic. This improves readability and allows easier modification if the SDK version changes in the future (although unlikely).

Hardware Connection and Initialisation

During this sprint, direct work in the laboratory environment became essential. Unlike previous stages, hardware behaviour could not be reliably simulated.

The following steps were validated in the lab:

- Physical connection of the monochromator and CCD camera
- Correct SDK recognition of the hardware
- Successful devices initialisation
- Stable communication during repeated acquisitions

Frequent on-site testing was required to verify real device responses, confirm correct implementation of the required subroutines based on the provided example code snippets, and ensure that measurements were successfully completed (correct timing, no communication losses).

Spectral Window Handling and Multi-Window Acquisition

One important technical limitation of the spectrometer is that a single acquisition window cannot cover the entire desired spectral range. Therefore, measurements must be performed in multiple adjacent spectral windows.

However, the monochromator takes the **centre wavelength** as input parameter. The task was therefore to calculate centres of each spectral window to cover desired total spectral range without gaps and overlapping. Stakeholders requested to define experiment spectral range using the **starting wavelength** and the desired **number of spectral windows**.

To improve usability, an additional function was implemented within the application to automatically calculate the centre wavelength for each required spectral window.

The user provides:

- The starting wavelength of the desired spectral range
- The number of spectral windows to acquire

The system then:

1. Calculates the centre wavelength for the first window.
2. Determines subsequent centre wavelengths sequentially so that each window follows directly after the previous one.
3. Ensures continuous spectral coverage without big gaps. The minimal gap between the next and previous windows corresponds to the spectral distance between last and second last pixel of the latter to preserve continuity of acquired data.

This abstraction simplifies experiment setup for researchers and reduces the possibility of configuration errors.

Real Acquisition Workflow

With SDK integration completed, the system now performs real acquisitions instead of simulated responses.

The workflow is as follows:

I. The application reads parameters from the GUI.
II. The requested number of acquisitions is realised in the custom subroutine *TakeSpectrum*().
III. For each spectrum:
    1. The centre wavelengths are calculated.
    2. Background spectrum acquired (as shutter is closed, the exact position of monochromator does not matter).
    3. Monochromator is instructed to move to the first centre wavelength.
    4. The requested number of acquisitions is triggered with data collected after each of them.
    5. The collected data is averaged, and background signal is subtracted.
    6. The measurement result is retrieved and accumulated.
    7. Monochromator is instructed to go the next centre wavelength, and the sequence repeated until full spectral range is covered.

8. Full data collected is then saved as file.
9. If saving is successful *TakeSpectrum()* returns true.

Instead of receiving a simulated “DONE” message, the system now waits for actual hardware confirmation that the measurement has completed successfully.

This represents a major milestone in the project, as the AMTCS system now controls real laboratory hardware in a stable and repeatable way.

Conclusion and MVP Progress at the End of the Sprint 4

By the end of Sprint 4, the spectrometer SDK has been successfully integrated into the C++ application, and stable hardware communication with the spectrometer (monochromator and CCD camera) has been achieved. Multi-spectral-window acquisition is implemented and real measurement data can now be obtained under laboratory conditions.

At this stage, the system is capable of executing a complete automated measurement cycle, combining temperature control, experiment sequencing, and spectrometer triggering. This means that the core functional components of the defined MVP are now in place.

However, the current orchestration layer was implemented in a minimal form to support early integration and testing. The next sprint will focus on improving system orchestration, adding structured logging, and strengthening error handling to ensure more robust long-term operation.

12. Sprint 5 – System Orchestration, Logging and Error Handling

Similarly to the previous sprint, during this one, a significant portion of the work required direct interaction with real hardware, which involved extensive in-lab testing and iterative debugging to verify system behaviour under actual operating conditions.

During the previous sprints, while working on individual components of the automation system, orchestration logic was considered and partially implemented across different parts of the project. By the beginning of Sprint 5, the system was already operational as a whole, with a basic orchestration layer allowing all components to work together.

However, this initial implementation was designed for controlled conditions, where the sequence of actions was known in advance (and adhered to at debug stage). The system is intended for use by uninformed users, and therefore must support intuitive operation while preventing invalid actions. This required introducing stricter control over user interaction, improving system robustness, and ensuring reliable behaviour under various conditions.

Another important goal of this sprint was to move from development-oriented execution (running components separately from different IDEs) to a more complete application lifecycle, including coordinated startup and graceful shutdown of all system components.

Startup

At startup, the C++ application checks whether the PLC is already running on RPi. If not, it initiates the PLC process.

Upon startup, the PLC main thread launches two helper threads (TCP and serial listeners), as described in Section 9. By default, the PLC operates in a waiting mode, expecting incoming commands via the TCP queue.

Graceful Shutdown

When the application exits:

1. It checks whether the PLC is running
2. If active, it sends a **"shutdown"** command
3. Waits for confirmation before closing

Upon receiving this command, the PLC:

1. Propagates the shutdown signal to communication threads
2. Stops the temperature controller
3. Terminates all threads in a controlled manner

This ensures a clean and safe shutdown of the entire system.

Orchestration Structure

The orchestration logic was divided into three main stages:

- Pre-experiment stage
- Experiment stage

Pre-Experiment Stage

The C++ application then performs the following steps (triggered by user interaction):

- Initialises the monochromator and CCD detector
- Sends a “ping” request to the PLC

Upon receiving the ping, the PLC:

- Confirms its availability
- Switches to active mode (non-blocking, does not wait if the queue is empty)
- Checks whether the TC is powered on
- If available:
    1. Reads the current temperature
    2. Sends it to the C++ application
    3. Sets it as the initial target
    4. Enables temperature control mode

If all checks are successful, the PLC reports readiness to the application and returns to waiting mode.

At this point, the application verifies that all required devices are connected and informs the user that the system is ready. The user can then proceed to the **Experiment Settings** tab.

This completes the pre-experiment stage. During this phase, no experiment execution takes place, allowing the user to safely configure all required parameters.

Experiment Stage

When the user presses **“Validate and Start”**, the application performs validation of input parameters:

- The temperature list is cleaned (duplicates removed, values sorted)
- If necessary, the order is reversed to start from the temperature closest to the current one

The validated parameters are stored in a dedicated object, serialized into a JSON string, and sent to the PLC. Additional parameters include:

- Total number of cycles (equal to the number of temperatures)
- Current cycle index (initialised to -1)

These parameters define the experiment state on the application side and are intended to support future recovery mechanisms (e.g., resuming after a crash), although this functionality is beyond the MVP scope.

At this point, the system transitions to the experiment execution.

Experiment Execution Logic (PLC Side)

During the experiment phase, the PLC operates in active mode and executes logic based on the experiment state (see Section 10).

Each cycle consists of two main steps:

Temperature Control

- *T.WAITING*
  A new target temperature is sent to the TC. After a delay, the PLC requests confirmation and compares it with the expected value. If matched, the state changes to *T.REQUESTED*.
- *T.REQUESTED*
  The PLC periodically reads the current temperature and compares it with the target.
  If the temperature remains within a ±0.5°C range for a defined number of readings, it is considered stabilised.
  Otherwise, the stability counter is reset to account for possible overshoots or undershoots of temperature.
- *T.COMPLETED*
  The PLC logs that the temperature has been reached and transitions to spectrum acquisition.

Spectrum Acquisition

- *S.WAITING*
  A TCP request is sent to the C++ application to start spectrum acquisition. Upon acceptance, the state changes to *S.REQUESTED*.
- *S.REQUESTED*
  The PLC waits for a "measurement completed" signal from the application.
- *S.COMPLETED*
  The PLC logs completion and proceeds to the next cycle. If all cycles are completed, the experiment ends.

C++ Application Behaviour

During the experiment phase, the C++ application operates as an execution unit:

- Waits for acquisition requests from the PLC
- Performs spectrum acquisition
- (Optionally) processes data (planned for future work)
- Saves results to a user-defined location
- Sends completion confirmation back to the PLC

After the experiment is completed, the application allows the user to start a new experiment or exit.

User Interface and User Experience (UX) Improvements

In earlier stages, all UI tabs were enabled simultaneously for debugging purposes.

In this sprint, the interface was restructured to guide the user through the workflow:

- At startup, only the **Connectivity** tab is enabled
- After successful hardware checks, it is disabled and **Experiment Settings** becomes active
- Once the experiment starts, **Experiment Settings** is disabled and **Experiment Flow** is enabled

This prevents invalid user actions and enforces correct operation order.

Logging and Progress Tracking

Logging was standardised and unified across the system.

- Log containers were extended to include timestamps automatically
- Messages were structured to reflect system state transitions
- A progress bar was implemented to visualise overall experiment progress

These improvements provide better visibility of system behaviour (e.g. Figure 8) and simplify debugging and monitoring.

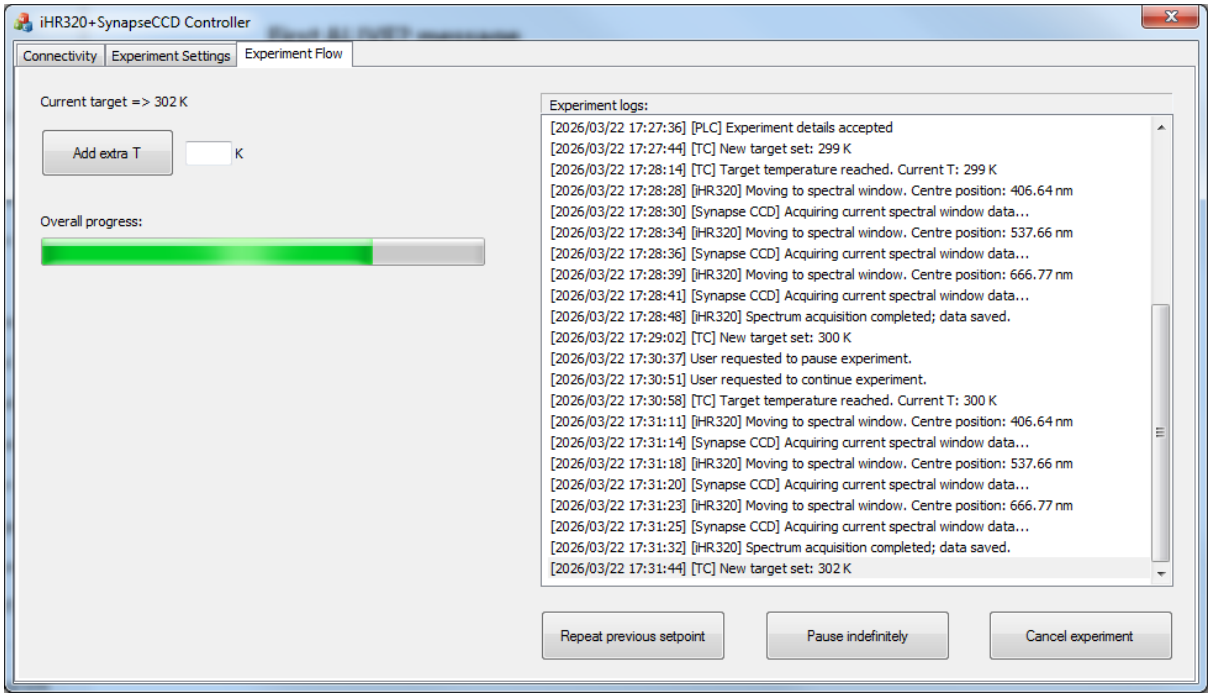


Figure 8. – 'Experiment Flow' screen showing typical logs during experiment execution.

Conclusion and MVP Progress at the End of Sprint 5

By the end of Sprint 5, system orchestration was structured into clear pre-experiment and experiment stages, with defined startup and shutdown subroutines ensuring coordinated execution and safe system termination.

At this stage, the application could perform a complete automated measurement cycle within a controlled and user-guided workflow. This represents a significant improvement over the minimal orchestration implemented in earlier sprints.

The MVP was functionally complete. Further work in sprint 6 focussed on improving robustness, strengthening error handling, and refining logging and usability for reliable long-term operation.

13. Sprint 6 – System Testing, Validation and Refinement

The final sprint focused on system testing, validation, and refinement under real laboratory conditions. Although the MVP had already been achieved in the previous sprint, this stage aimed to improve system robustness, usability and overall reliability.

Extended Experiment Control

During this sprint, extended 'experiment flow'-time controls were introduced, including *pause/resume experiment, cancel experiment*, *repeat previous temperature datapoint*, and *add an additional temperature datapoint*.

Although non-essential for basic automation, they enhanced user control and flexibility. In addition, visualisation of the current target temperature was added to the UI to provide clearer system status feedback.

System Recovery and Fault Tolerance

To improve overall system robustness, basic recovery mechanisms were implemented. During development, it became clear that the system should be able to recover from unexpected failures of individual components (PLC or C++ application, but not both simultaneously).

A heartbeat mechanism was introduced, allowing the main components to exchange status signals periodically. If the PLC became unresponsive, the application could restart it. The PLC startup sequence was extended to request the current experiment state (if available). Similarly, if the application crashed and was restarted by user, it detected whether the PLC is already running and requested the experiment state after completing the hardware initialisation (Figure 9).

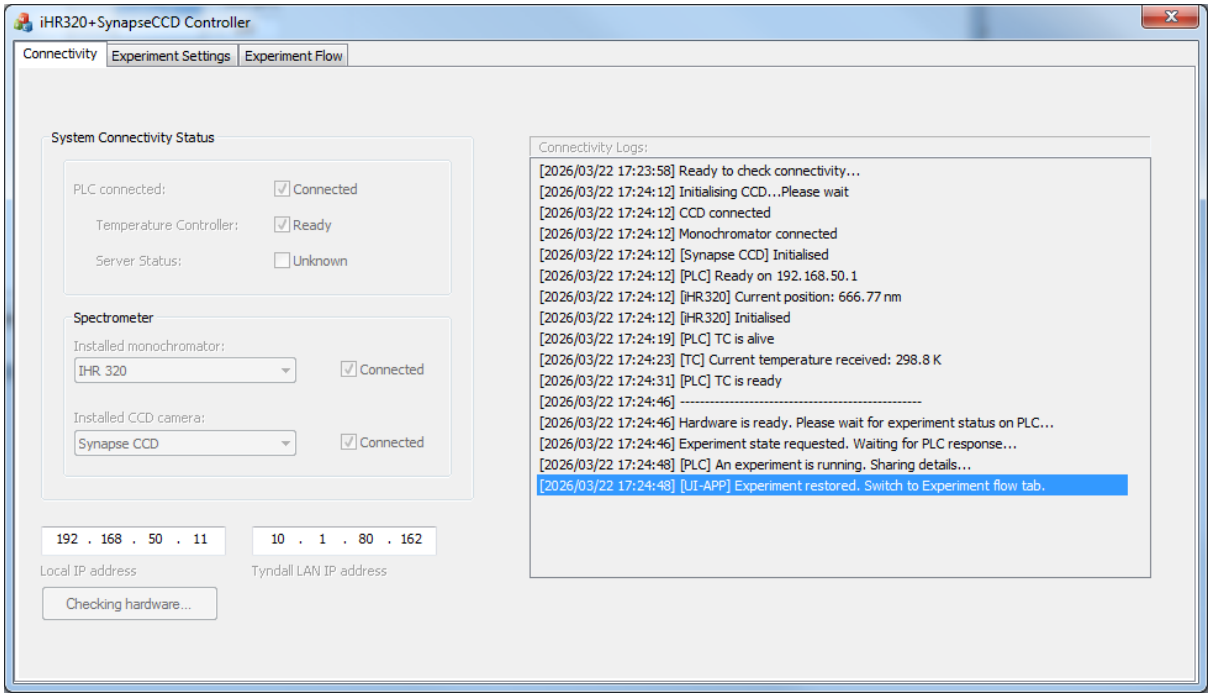


Figure 9. – Logs after UI App recovery (simulated crash).

This functionality improves system fault tolerance and supports more reliable long-term operation.

Cosmic Ray Artefact Removal

Cosmic ray artefacts distort spectroscopic data by introducing abnormal intensity spikes. In manual experiments, researchers could identify and correct them, but in automated operation only averaged data is available.

To address this, a pixel-wise comparison approach was implemented. When multiple acquisitions are available (minimum of three), abnormal values are detected and replaced using average of unaffected data points. This improves the reliability and quality of the final measured spectra.

Auto exposure feature

One of important parameters of a spectroscopic measurement is its dynamic range, i.e. how wide the measured signal spans in intensity. To maximise it, the maximum signal should be as close as possible to the detector's full scale without entering saturation region.

The CCD camera used in the lab has a 15-bit detector, with a maximum signal of 32767 counts. However, based on discussions with lab users, it was identified that the detector becomes non-linear when the signal exceeds approximately 50% of this range.

During temperature-resolved experiments, signal intensity can change significantly. While it is usually preferable to keep acquisition settings fixed, this is not always possible in practice. This need for adjusting settings during experiments was also pointed out by stakeholders.

Based on their suggestions, the following approach was implemented. If the signal exceeded 50% of the detector range, exposure time and slit width were reduced the signal to ~10%. If the signal dropped below 6%, the settings were increased it to ~37.5%. This asymmetric approach was suggested to reduce how often adjustments are needed during experiments.

Whenever the signal was outside the desired range, the current measurement was saved (with setting count number), new acquisition parameters were calculated, setting count number incremented and the measurement was repeated (see logs in Figure 10). This process was applied iteratively until the signal returned to the acceptable range or hardware/user defined limits were reached.

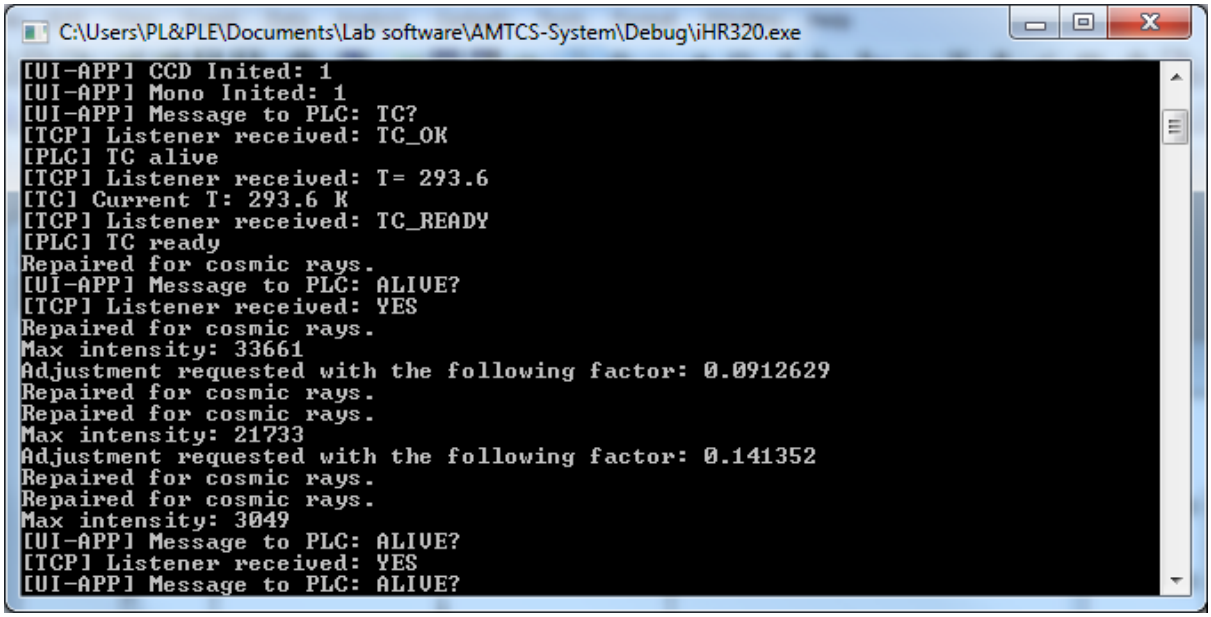


Figure 10. – Example logs during measurement subroutine (UI App terminal).

Testing and Refinement

Throughout the sprint, continuous testing was performed under varied scenarios, including cases where individual pieces of equipment were disconnected or powered-off. Identified issues were resolved, and the system was further stabilised.

It was observed that lab users testing the system manually switched off monitors to eliminate parasitic light during measurements. To reduce manual intervention, this step was automated by introducing programmatic control of monitor power before and after spectrum acquisition.

Another observation was that the system stopped working reliably after Windows entered sleep mode. To prevent this, appropriate execution state attributes were assigned to the C++ application UI thread at the start of an experiment, ensuring that the system does not enter sleep mode while in experiment.

Finally, the codebase was refined and prepared for the first release.

Conclusion and MVP Progress at the End of Sprint 6

Although MVP was achieved in Sprint 5, Sprint 6 focused on validation, robustness, and usability improvements. The system was tested under varied conditions, errors were resolved, and additional functionality ensured reliable operation in real laboratory use.

Conclusion

The project was successfully completed. The main objectives were achieved: the system was automated, tested, and demonstrated stable operation in laboratory conditions.

The developed solution enables automated temperature-dependent measurements, reduces the need for manual intervention, and improves experiment repeatability and efficiency.

Reflection

The idea for this project came from my internship, when I learned that the laboratory had a long-standing need to automate their experimental setup. This project became a real opportunity to apply the knowledge gained during my studies in a practical environment.

In particular, I was able to apply and develop skills in programming, system design, and communication between hardware components. It was also my first experience working directly with real laboratory equipment and integrating multiple devices into one system.

The development process was iterative, starting with simulation and gradually moving to real hardware. This approach helped identify problems early and reduce integration risks.

Overall, the project provided valuable hands-on experience and a better understanding of how software is applied in real engineering systems.

Key Skills

During the project, the following skills were developed and improved:

- C++ and Python programming and application development
- Basic multithreading concepts
- Working with equipment manufacturer SDK and instrument control
- Serial port and TCP communications
- Hardware–software integration
- Debugging and system testing
- Version control (Git) across multiple environments (PC, RPi, VMs)

Challenges

One of the main challenges was coding in C++ (MFC, COM, multithreading). Since I had no previous exposure to C++, this required a steep learning curve. In particular, understanding event-driven programming and integrating COM-based SDK components into the application required substantial effort.

Another challenge was working with legacy systems, including an older Windows environment and SDK that required outdated development tools. For example, differences between documented SDK behaviour and actual system behaviour required additional testing and adjustments.

Integration of devices from different manufacturers was also challenging, as each device used different communication methods. This required careful coordination of serial and TCP communication, as well as synchronisation of system components during experiment execution.

In addition, the project required developing an understanding of the experimental workflow and laboratory requirements. This included learning how temperature stabilisation, spectrum acquisition, and data quality affect experimental results. Close collaboration with laboratory staff was essential to correctly interpret these requirements and translate them into system logic.

These challenges were addressed through continuous testing, iterative development, and regular consultation with Tyndall mentor and laboratory users.

Future Work

At the planning stage, the project focused on achieving the MVP, while also identifying possible future improvements.

The project will continue beyond submission, as the internship period extends for an additional month. This provides an opportunity to further improve the system based on user feedback from real remote (rather than in-person) experiments.

It is worth noting that some features initially identified as future work were implemented already within the scope of this project. In particular, cosmic ray artefact removal was developed and integrated into the automated data processing. This functionality also enabled the implementation of the auto-exposure feature, which had originally been planned as future work but was successfully completed during the project.

Two further developments are planned:

- Development of a server-based logging and notification system (hosted on RPi). This functionality has been partially implemented but is not yet integrated.
- Mobile and/or browser-based interface for remote experiment monitoring.

A mobile or web-based interface would be particularly useful, as this automation allows researchers to avoid being physically present in the laboratory during long experiments. Therefore, enabling remote experiment monitoring would be a logical next step. However, its implementation requires additional time and consideration of institutional network security requirements and is therefore beyond the scope of this project.

Another important future step is the preparation of user documentation and provision of training for laboratory users.

References

Cryogenic Control Systems Inc. (2011). *User's Guide Model 32 & 32B Cryogenic Temperature Controller*.

HORIBA Scientific (2026). *HORIBA EzSpec-SDK*. [online] GitHub. Available at: https://github.com/HORIBAEzSpecSDK [Accessed 15 Feb. 2026].

Lohmann, N. (2021). *JSON for Modern C++ version 3.10.0*. [online] GitHub. Available at: https://github.com/nlohmann/json/releases/tag/v3.10.0 [Accessed 15 Feb. 2026].

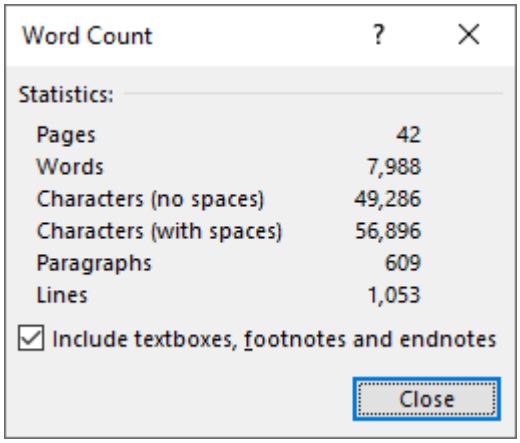
National Instruments (2023). *LabVIEW Instrument Design Libraries for High-Speed Serial Instruments Download*. [online] Ni.com. Available at: https://www.ni.com/en/support/downloads/drivers/download.labview-instrument-design-libraries-for-high-speed-serial-instruments.html [Accessed 15 Feb. 2026].

OE-FET (2020). *GitHub - OE-FET/customxepr: Instrument control suite for Bruker E500, MercuryiTC and Keithley 2600*. [online] GitHub. Available at: https://github.com/OE-FET/customxepr [Accessed 15 Feb. 2026].

Oxford Instruments (2026). *Andor Software Development Kit (SDK)* . [online] OXFORD Instruments | ANDOR. Available at: https://andor.oxinst.com/products/software-development-kit/software-development-kit [Accessed 15 Feb. 2026].

Thorlabs Inc. (2026). *Thorlabs Official Account and Source for Example Code*. [online] GitHub. Available at: https://github.com/Thorlabs [Accessed 15 Feb. 2026].

Wikipedia Contributors (2024). *Single-event upset*. [online] Wikipedia. Available at: https://en.wikipedia.org/wiki/Single-event_upset [Accessed 15 Feb. 2026].

Witter, A. (2004). *SynerJY SDK* [PDF version].

Zubialevich M. (2026). *AMTCS-System (branch: spike/rpi-pc-tcp-connectivity-test)*. [online] GitHub. Available [on request] at https://github.com/ZuMaVla/AMTCS-System/tree/spike/rpi-pc-tcp-connectivity-test